FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
UNIVERSIDAD DE CHILE

Go-Back-N y control de congestión
TCP simplificado con mecanismos de TCP Tahoe

Informe de Actividad — Módulo 3: Protocolos de transporte

Autor: Felipe Arriagada, Vicente Cepeda

Curso: CC4303 Redes — sección Ivana Bachmann

Fecha: 18 de octubre de 2026

Índice

1. Declaración sobre uso de IA / LLMs **2**

2. Introducción **2**

3. Marco Conceptual **3**

3.1. Ventana deslizante y Go-Back-N . . . 3

3.2. Control de congestión en TCP Tahoe . . . 3

4. Implementación **4**

4.1. Estructura general . . . 4

4.2. Parte 1: la clase `CongestionControl` . . . 4

4.3. Separación de Stop & Wait y selección del modo . . . 5

4.4. Parte 2: preguntas previas . . . 5

4.5. Receptor Go-Back-N . . . 6

4.6. Emisor Go-Back-N . . . 6

4.7. Parte 3: Go-Back-N con control de congestión . . . 7

4.8. Modo debug . . . 10

5. Pruebas **10**

5.1. Test de la clase `CongestionControl` . . . 10

5.2. Go-Back-N sin pérdidas . . . 11

5.3. Go-Back-N con pérdidas . . . 11

5.4. Control de congestión sin pérdidas . . . 12

5.5. Control de congestión con pérdidas . . . 12

5.6. Comparación con y sin control de congestión . . . 14

6. Situaciones de borde y limitaciones **16**

7. Conclusiones **17**

Referencias **18**

1. Declaración sobre uso de IA / LLMs

Para el desarrollo de esta actividad se utilizó Claude (Anthropic, modelo Claude Opus 5.5) como tutor y apoyo tanto en el código como en la elaboración del informe. El trabajo se realizó paso a paso: el modelo explicaba cada concepto preguntaddo y los integrantes del grupo lo implementaban, probaban y corregían. En concreto, el modelo se usó para:

(a) explicar el funcionamiento de slow start y congestion avoidance, y guiar la implementación de la clase `CongestionControl`, que fue escrita por el grupo; el modelo detectó errores en ella (métodos indentados dentro del constructor, una condición innecesaria en `event_timeout` y `cwnd` reiniciado en 1 byte en lugar de 1 MSS),

(b) detectar errores en la integración: una función duplicada y la pérdida de la llamada a `move_window`, que provocaba un ciclo infinito,

(c) escribir los programas de prueba (los tests en modo Go-Back-N, el cliente y el servidor para enviar archivos, y `experimento.py`) y apoyar el uso de `netem`,

(d) interpretar las trazas del modo debug y organizar y redactar el informe, incluidos los diagramas y el gráfico.

Todas las salidas y resultados incluidos en este informe provienen de ejecuciones reales del código en el computador de uno de los integrantes.

2. Introducción

En la actividad anterior se implementó una versión simplificada de TCP sobre UDP que usaba **Stop & Wait**: el emisor envía un segmento y no envía el siguiente hasta recibir su confirmación. Ese esquema es confiable, pero desperdicia la capacidad de la red, ya que durante cada tiempo de ida y vuelta (RTT) solo hay un segmento en tránsito (Kurose y Ross, 2021).

En esta actividad (Equipo docente CC4303, 2026) se reemplaza Stop & Wait por **Go-Back-N**, un protocolo de ventana deslizante que permite tener varios segmentos enviados y aún no confirmados. Luego se agrega un mecanismo de **control de congestión** inspirado en TCP Tahoe, que ajusta dinámicamente el tamaño de esa ventana según el estado de la red: la agranda mientras los segmentos llegan y la reduce drásticamente cuando detecta pérdidas. Como indica el enunciado, la implementación sigue siendo una versión simplificada de TCP, que solo incluye slow start y congestion avoidance.

El trabajo se divide en tres partes: (1) una clase `CongestionControl` que encapsula la lógica de control de congestión, (2) las funciones `send_using_go_back_n` y `recv_using_go_back_n`, con ventana fija, y (3) la integración de ambas, de modo que el tamaño de la ventana de Go-Back-N lo determine el control de congestión.

La sección 3 presenta los conceptos necesarios. La sección 4 describe la implementación y las decisiones de diseño, e incluye las respuestas a las preguntas de la parte 2. La sección 5 documenta las pruebas pedidas, con y sin pérdidas, y la comparación entre Go-Back-N con y sin control de congestión. La sección 6 reúne las situaciones de borde encontradas y la sección 7 sintetiza los hallazgos.

3. Marco Conceptual

3.1. Ventana deslizante y Go-Back-N

En un protocolo de **ventana deslizante**, el emisor puede tener hasta N segmentos enviados y no confirmados. Esos segmentos forman la *ventana*: cuando llega la confirmación del más antiguo, la ventana "se desliza" hacia adelante y entra un segmento nuevo, que se envía de inmediato. Así, en lugar de un segmento por RTT, se pueden enviar hasta N.

En **Go-Back-N** (Kurose y Ross, 2021):

- El **receptor** solo acepta el segmento que espera, en orden. Si llega uno fuera de orden, lo descarta y responde con el ACK del último segmento recibido en orden (**ACK acumulativo**). No guarda segmentos ni usa temporizadores.
- El **emisor** usa un único temporizador para toda la ventana. Cada ACK confirma todos los segmentos hasta el número que indica. Si el temporizador expira, el emisor **reenvía la ventana completa**; de ahí el nombre del protocolo.

Stop & Wait corresponde al caso particular de Go-Back-N con $N = 1$. La desventaja de Go-Back-N es que, ante una sola pérdida, también se retransmiten los segmentos posteriores, aunque el receptor ya los hubiera recibido. Selective Repeat evita esto guardando en el receptor los segmentos fuera de orden y usando un temporizador por segmento, a costa de mayor complejidad.

3.2. Control de congestión en TCP Tahoe

El control de errores protege a la comunicación de las pérdidas, pero no evita provocarlas: si todos los emisores envían lo más rápido posible, los enrutadores se saturan y descartan paquetes. El **control de congestión** hace que cada emisor regule su tasa de envío según las pérdidas que observa (Jacobson, 1988; Allman, Paxson y Blanton, 2009). Además de proteger la red, esto la hace **justa** (*fair*): ningún flujo acapara la capacidad compartida.

TCP mantiene una **ventana de congestión** (`cwnd`), medida en bytes, que limita cuántos datos pueden estar en tránsito. Su valor depende de dos estados:

- **Slow start.** La conexión parte con `cwnd` = 1 MSS (*Maximum Segment Size*) y suma 1 MSS por cada ACK. Como en cada ronda llegan tantos ACKs como segmentos hay en la ventana, `cwnd` **se duplica en cada RTT**: el crecimiento es exponencial. El nombre "lento" se debe solo a que parte desde un segmento.
- **Congestion avoidance.** Cuando `cwnd` alcanza el umbral `ssthresh`, cada ACK suma solo MSS/k, con k la cantidad de MSS que caben en la ventana. Así, `cwnd` crece 1 MSS por cada ventana completa confirmada: el crecimiento es lineal (*additive increase*).

Ante un **timeout**, TCP Tahoe asume congestión severa: fija `ssthresh = cwnd/2` (*multiplicative decrease*), vuelve `cwnd` a 1 MSS y regresa a slow start. Tahoe también incluye *fast retransmit*, que esta actividad no implementa.

4. Implementación

4.1. Estructura general

Se trabajó sobre una copia del código de la actividad de Stop & Wait. La solución queda en los siguientes archivos:

- `CongestionControl.py`: la clase `CongestionControl` (parte 1).
- `SocketTCP.py`: la clase `SocketTCP`, ahora con Stop & Wait y Go-Back-N, con y sin control de congestión (partes 2 y 3).
- `socketUDP.py` y `slidingWindowCC.py`: las clases entregadas por el equipo docente, que se importan sin modificaciones.
- Programas de prueba: `test_client_gbn.py` y `test_server_gbn.py` (tests de `send` y `recv`), `cliente_gbn.py` y `servidor_gbn.py` (envío de un archivo) y `experimento.py` (comparación con y sin control de congestión).

4.2. Parte 1: la clase `CongestionControl`

La clase es una **máquina de estados** independiente de los sockets: no envía ni recibe nada, solo reacciona a dos eventos, "llegó un ACK" y "ocurrió un timeout", que le informa quien la usa. Su constructor recibe el MSS e inicializa las variables pedidas: el estado parte en slow start, `cwnd` en 1 MSS (en bytes) y `ssthresh` en `None`, porque solo se define tras el primer timeout.

```
class CongestionControl:
    def __init__(self, MSS: int):
        self.MSS = MSS
        self.current_state = "slow_start"  # partimos en slow start
        self.cwnd = MSS                    # partimos en 1 MSS (en bytes)
        self.ssthresh = None               # se define en el primer timeout

    def get_cwnd(self):
        return self.cwnd

    def get_MSS_in_cwnd(self):
        return int(self.cwnd // self.MSS)

    def get_ssthresh(self):
        return self.ssthresh

    def is_state_slow_start(self):
        return self.current_state == "slow_start"

    def is_state_congestion_avoidance(self):
        return self.current_state == "congestion_avoidance"

    def event_ack_received(self):
        if self.is_state_slow_start():
            self.cwnd += self.MSS
            if self.ssthresh is not None and self.cwnd >= self.ssthresh:
                self.current_state = "congestion_avoidance"
        elif self.is_state_congestion_avoidance():
```

```
            self.cwnd += self.MSS * (1 / self.get_MSS_in_cwnd())

    def event_timeout(self):
        self.ssthresh = self.cwnd // 2
        self.cwnd = self.MSS
        self.current_state = "slow_start"
```

Algunas decisiones de diseño:

- **`cwnd` en bytes.** Por eso se inicializa en `MSS` y no en 1, y `get_MSS_in_cwnd` usa división entera para contar cuántos segmentos *completos* caben (con `cwnd` = 17 y MSS = 4, retorna 4).
- **`ssthresh = None` antes del primer timeout.** Mientras no hay umbral, slow start crece sin límite. La condición `ssthresh is not None` es necesaria además porque en Python comparar un número con `None` lanza `TypeError`.
- **`cwnd` puede ser decimal.** En congestion avoidance cada ACK suma una fracción de MSS (por ejemplo, $8/3 \approx 2{,}67$). Se decidió mantener `cwnd` como `float` para no perder esas fracciones: si se truncaran, la ventana podría estancarse. `get_MSS_in_cwnd` retorna igualmente un entero.
- **El orden en `event_timeout`.** `ssthresh` se calcula *antes* de reducir `cwnd`, porque debe ser la mitad de la ventana que había al momento de la congestión.
- **El cambio de estado solo ocurre en slow start.** Con un ACK nunca se sale de congestion avoidance; solo un timeout devuelve a slow start.

4.3. Separación de Stop & Wait y selección del modo

Como pide el enunciado, el código de Stop & Wait se movió a `send_using_stop_and_wait` y `recv_using_stop_and_wait`, y las funciones `send` y `recv` quedaron como selectoras del protocolo según el parámetro `mode`, con Stop & Wait como valor por defecto:

```
def send(self, message, mode="stop_and_wait"):
    if mode == "stop_and_wait":
        self.send_using_stop_and_wait(message)
    elif mode == "go_back_n":
        self.send_using_go_back_n(message)

def recv(self, buff_size, mode="stop_and_wait"):
    if mode == "stop_and_wait":
        return self.recv_using_stop_and_wait(buff_size)
    elif mode == "go_back_n":
        return self.recv_using_go_back_n(buff_size)
```

El código sugerido por el enunciado para `recv` no incluye `return`; sin él, `recv` retornaría siempre `None`, aunque los datos hubieran llegado correctamente.

4.4. Parte 2: preguntas previas

(1) ¿Por qué sirve como base el código de Stop & Wait? ¿Qué similitud tiene con Go-Back-N? Stop & Wait corresponde a Go-Back-N con una ventana de tamaño 1, por lo que

su lógica es la misma: el emisor envía, espera ACKs y, si ocurre un timeout, retransmite. En ambos hay un solo temporizador y el receptor solo acepta el segmento que espera, en orden. La diferencia es que Go-Back-N permite tener hasta N segmentos en tránsito sin confirmar y, ante un timeout, reenvía toda la ventana. Por ello se puede reutilizar la estructura de envío, la espera de ACKs y el manejo del timeout, extendiéndola a una ventana de mayor tamaño. El propio ejemplo del enunciado lo muestra: implementa Stop & Wait con `SlidingWindowCC(1, ...)`, una ventana de tamaño 1.

(2) ¿Qué ocurre con la función `recv`? ¿Cambia con el uso de `SocketUDP` y `SlidingWindowCC`? La función `recv` cambia poco. El receptor de Go-Back-N no necesita temporizadores, porque nunca retransmite por iniciativa propia: las pérdidas las detecta el temporizador del emisor. Por eso no necesita `SocketUDP`. Tampoco necesita `SlidingWindowCC`, porque solo acepta el segmento que espera (su ventana es de tamaño 1) y descarta los que llegan fuera de orden. Lo que el receptor debe cumplir es (a) responder a un segmento fuera de orden o repetido con el ACK del **último segmento recibido en orden** (ACK acumulativo), en lugar de confirmar lo que acaba de llegar, y (b) avanzar el número de secuencia esperado **en bytes**, porque `SlidingWindowCC` numera los segmentos según su largo.

4.5. Receptor Go-Back-N

Al revisar la función `recv_data` de la implementación de Stop & Wait se comprobó que **ya cumplía los requisitos de un receptor Go-Back-N**, por lo que no requirió cambios:

```
def recv_data(self):
    self.socket.settimeout(None)
    while True:
        seg, _ = self.recive_segment()
        if seg["SYN"] or seg["ACK"] or seg["FIN"]:
            continue
        if seg["SEQ"] == self.seq:                          # solo acepta el
    esperado
            self.seq = self.seq + len(seg["DATA"])          # avanza en bytes
            self.send_segment(self.new_segment(self.seq, ack = True))
            return seg["DATA"]
        debug("dato repetido, reenviando ACK %d" % self.seq)
        self.send_segment(self.new_segment(self.seq, ack = True))  # ACK
    acumulativo
```

Esto se debe a que el receptor de Stop & Wait ya se había implementado de la forma general: el ACK indica el próximo byte esperado, y ante un segmento repetido o fuera de orden se repite ese mismo ACK. Ese comportamiento es correcto tanto con uno como con N segmentos en tránsito. Por lo tanto, `recv_using_go_back_n` reutiliza la misma lógica, incluido el manejo de `buff_size` de la actividad anterior:

```
def recv_using_go_back_n(self, buff_size):
    return self.recv_using_stop_and_wait(buff_size)
```

4.6. Emisor Go-Back-N

Ventana. El mensaje se divide en una lista cuyo primer elemento es su largo (igual que en Stop & Wait) y el resto son trozos de datos. Con esa lista se crea un objeto `SlidingWindowCC` con

número de secuencia inicial `self.seq`, que asigna a cada trozo su número de secuencia según el largo en bytes de los anteriores. En la parte 2 la ventana tenía un tamaño fijo de 8 segmentos y trozos de 16 bytes; en la parte 3 los trozos pasaron a ser de MSS = 8 bytes.

Uso de `SocketUDP` sobre el socket existente. `SocketUDP` crea internamente su propio socket UDP, pero la conexión (puerto local, dirección de la contraparte y estado del handshake) vive en el socket de `SocketTCP`. Si `SocketUDP` enviara desde su propio socket, lo haría desde otro puerto y el receptor nunca recibiría los datos. Por ello se decidió **reemplazar el socket interno de `SocketUDP` por el de la conexión**, sin modificar la clase entregada:

```
udp = SocketUDP()
udp.socket_udp.close()           # se cierra el socket interno que no se usara
udp.socket_udp = self.socket     # y se usa el de la conexion
self.socket.setblocking(False)   # SocketUDP necesita un socket no bloqueante
udp.settimeout(TIMEOUT)
```

Al terminar el envío, el socket vuelve a modo bloqueante, ya que `recv`, `close` y las demás funciones lo esperan así.

Temporizador. Go-Back-N usa un solo temporizador (índice 0) para toda la ventana. Se encontraron dos detalles de `SocketUDP` que obligan a manejarlo con cuidado:

- Los temporizadores son objetos `Timer` de Python, que solo pueden iniciarse una vez. Tras un timeout hay que llamar a `stop_timer(0)` antes de reenviar, para que `sendto` cree un temporizador nuevo; de lo contrario se lanza `RuntimeError`.
- `stop_timer` falla si no hay temporizador, por lo que al terminar se verifica antes que exista.

Funcionamiento. El emisor envía primero toda la ventana. Luego, por cada ACK recibido, cuenta cuántos segmentos del inicio de la ventana quedan confirmados (los que cumplen seq + largo $\leq$ ACK, ya que el ACK indica el próximo byte esperado), desliza la ventana esa cantidad de posiciones y envía los segmentos que entraron. Un ACK que no confirma nada nuevo, como los ACK repetidos del receptor, se ignora. Si el temporizador expira, se reenvía la ventana completa. La figura 1 ilustra este comportamiento con una ventana de 4 segmentos de 10 bytes.

La figura muestra también la debilidad de Go-Back-N: los segmentos 20 y 30 llegaron al receptor, pero como venían fuera de orden fueron descartados y deben retransmitirse.

Caso borde del handshake. La actividad anterior resolvió la pérdida del último ACK del handshake dentro de `send`: si el emisor recibe un `SYN+ACK` después de terminar `connect`, significa que su ACK se perdió, y lo reenvía. Este manejo se conservó en `send_using_go_back_n`.

4.7. Parte 3: Go-Back-N con control de congestión

Nuevos atributos. Se agregaron a `SocketTCP` el booleano `congestion_control_on` (por defecto `True`) con su función *setter* `set_congestion_control`, el objeto `congestion_controler` y el contador `number_of_sent_segments`, que cuenta cada segmento de datos enviado y se usa en el experimento de la sección 5.6.

Inicialización. El mensaje se divide en trozos de MSS = 8 bytes. Si el control de congestión está activo, se crea un `CongestionControl(8)` y el tamaño inicial de la ventana se obtiene de él; si no, la ventana se mantiene fija en 8:

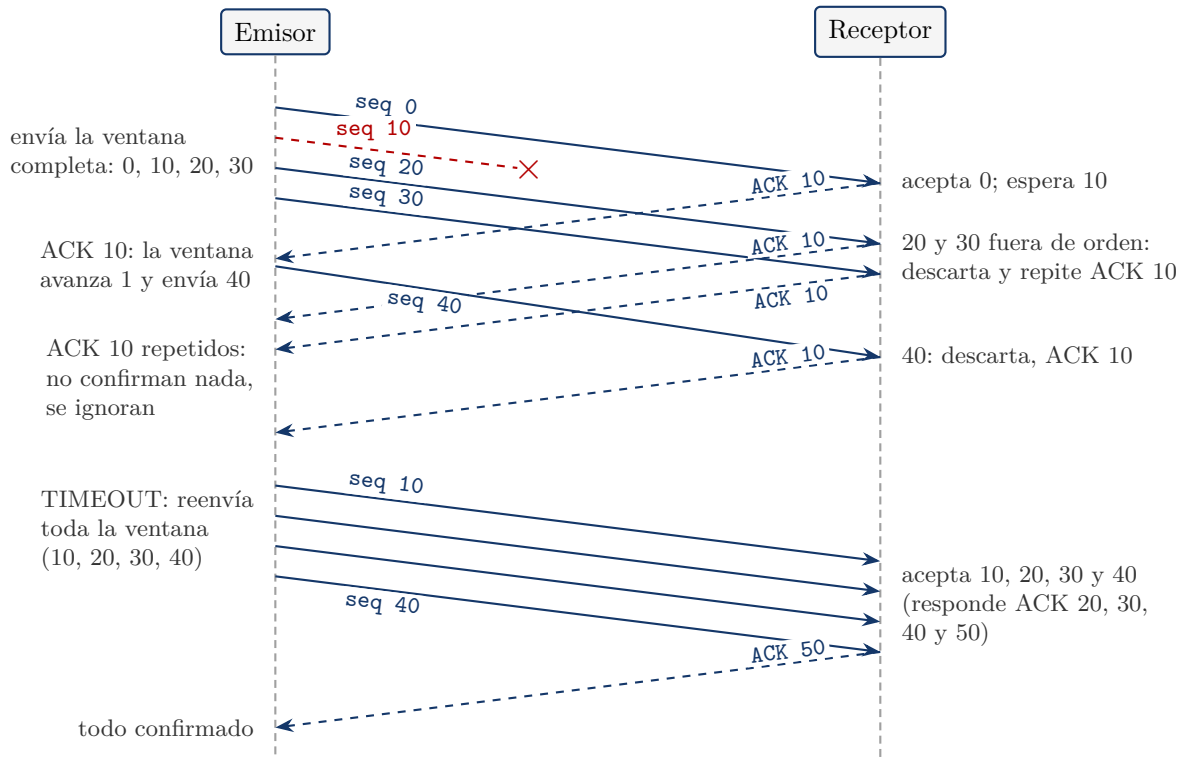


Figura 1: Go-Back-N con ventana de 4: pérdida del segmento con seq 10

```
MSS = 8
data_list = [str(len(msj)).encode()] + [msj[i:i+MSS] for i in range(0, len(msj),
    MSS)]
if self.congestion_control_on:
    self.congestion_controler = CongestionControl(MSS)
    window_size = self.congestion_controler.get_MSS_in_cwnd()
    self.debug_cc("inicio", window_size)
else:
    window_size = 8
window = SlidingWindowCC(window_size, data_list, self.seq)
```

¿Qué valor tiene `window_size` en este punto? Vale **1**: el objeto recién creado tiene `cwnd` = 1 MSS, como verifica el primer test de la clase (sección 5.1), por lo que `get_MSS_in_cwnd()` retorna 1. El modo debug lo confirma en la primera línea de cada envío (sección 5.4).

Timeout. Antes de reenviar, se informa el evento al control de congestión y la ventana se reduce al nuevo tamaño. Como el reenvío usa el tamaño *nuevo*, tras un timeout se reenvía un solo segmento en lugar de ocho:

```
except TimeoutError:
    debug("TIMEOUT, reenviando ventana")
    udp.stop_timer(0)
    if self.congestion_control_on:
        self.congestion_controler.event_timeout()
        window_size = self.congestion_controler.get_MSS_in_cwnd()
        window.update_window_size(window_size)
        self.debug_cc("timeout", window_size)
    for i in range(window_size):
```

```
        if window.get_data(i) is not None:
            enviar(i)
    continue
```

ACK y caso borde de la ventana que se reduce. El enunciado advierte que la ventana puede reducirse justo después de enviar todos sus segmentos, de modo que luego llegue un ACK mayor que el mayor número de secuencia de la ventana. Por ejemplo, si se enviaron 8 segmentos y un timeout reduce la ventana a 1, los ACK de esos 8 segmentos pueden seguir llegando. Para resolverlo, la ventana se desliza **de a una posición, mientras su primer segmento esté confirmado por el ACK**. Esto equivale a moverla como si hubiese llegado su último elemento, tantas veces como sea necesario, que es lo que pide el enunciado:

```
ack = res["SEQ"]
pasos = 0
while (window.get_data(0) is not None and
       window.get_sequence_number(0) + len(window.get_data(0)) <= ack):
    window.move_window(1)
    pasos += 1
if pasos == 0:
    continue        # ACK repetido: no confirma nada nuevo
self.seq = ack
```

Luego se informa el ACK al control de congestión y se actualiza el tamaño de la ventana. Los segmentos nuevos que hay que enviar provienen de dos fuentes: los que entraron porque la ventana se *deslizó* y los que entraron porque *creció*. Ambos quedan al final de la ventana, desde la posición `tamano_anterior` − `pasos`, y se envían de forma consecutiva:

```
tamano_anterior = window_size
if self.congestion_control_on:
    self.congestion_controler.event_ack_received()
    window_size = self.congestion_controler.get_MSS_in_cwnd()
    window.update_window_size(window_size)
    self.debug_cc("ack %d" % ack, window_size)
primero_nuevo = max(0, tamano_anterior - pasos)
nuevos = [i for i in range(primero_nuevo, window_size)
          if window.get_data(i) is not None]
if nuevos or window.get_data(0) is None:
    udp.stop_timer(0)
for i in nuevos:
    enviar(i)
```

El `max(0, ...)` cubre el caso borde anterior: si la ventana se deslizó más posiciones que su tamaño previo, todos sus segmentos son nuevos. El temporizador se reinicia solo si se envían segmentos nuevos (el `sendto` arranca uno nuevo) o si ya no queda nada por confirmar. Si quedan segmentos pendientes pero no entró ninguno nuevo, el temporizador **no** se detiene, porque ningún envío lo volvería a iniciar y el emisor podría quedar esperando indefinidamente. Sin control de congestión, `window_size` no cambia y este mismo código se comporta exactamente como el de la parte 2.

Decisiones de diseño.

- **Un `CongestionControl` por mensaje.** El objeto se crea al inicio de cada llamada a

`send`, por lo que cada mensaje comienza en slow start con `cwnd` = 1 MSS. El TCP real mantiene `cwnd` durante toda la conexión; se eligió esta opción porque hace que cada transferencia sea independiente y más fácil de analizar.

- **Un evento por ACK.** `event_ack_received` se llama una vez por cada ACK que confirma datos nuevos, aunque un ACK acumulativo confirme varios segmentos a la vez (por ejemplo, si se perdieron ACKs intermedios).

4.8. Modo debug

Además del modo debug de la actividad anterior, se agregó la función `debug_cc`, que tras cada evento que puede modificar `cwnd` imprime el evento, el estado, `cwnd`, `ssthresh` y el tamaño de la ventana:

```
def debug_cc(self, evento, window_size):
    cc = self.congestion_controler
    estado = "slow start" if cc.is_state_slow_start() else "congestion avoidance"
    debug("[CC]  %-12s estado=%-20s cwnd=%6.2f ssthresh=%s window_size=%d"
          % (evento, estado, cc.get_cwnd(), cc.get_ssthresh(), window_size))
```

Pérdidas. La simulación de pérdidas a mano de la actividad anterior (`LOSS_RATE`) actúa en `send_segment`, pero los datos de Go-Back-N se envían con `SocketUDP.sendto`, que no pasa por esa función. Por eso, todas las pruebas con pérdidas de este informe se realizaron con `netem` (Linux man-pages, 2026), que afecta por igual a ambas direcciones.

5. Pruebas

Salvo que se indique lo contrario, las pruebas con pérdidas usaron `netem` con 20 % de pérdida y 100 ms de retardo en cada dirección. Se usó un retardo menor al de la actividad anterior (0,5 s) porque, con el `TIMEOUT` de 1 s, un RTT de 1 s provocaría timeouts aun sin pérdidas. Antes de cada prueba se verificó que `netem` estuviera activo:

```
$ sudo tc qdisc add dev lo root netem loss 20.0% delay 0.1s
$ tc qdisc show dev lo
qdisc netem 8003: root refcnt 2 limit 1000 delay 100ms loss 20%
```

5.1. Test de la clase `CongestionControl`

Se ejecutó `CongestionControl_test.py` con MSS = 1, 2, 3, 4, 7, 8, 10, 13, 100 y 1000. En todos los casos se aprobaron los cinco tests:

```
$ python3 CongestionControl_test.py
Initial test passed
ACK test passed
Timeout test passed
ssthresh test passed
Timeout test 2 passed
```

El test recorre el ciclo completo de TCP Tahoe. Con MSS = 7: (1) el objeto parte con `cwnd` =

7 y `ssthresh = None`; (2) en tres rondas llegan 1, 2 y 4 ACKs, y la ventana se duplica en cada una ($1 \to 2 \to 4 \to 8$ MSS); (3) un timeout fija `ssthresh` $= 4$ MSS y vuelve `cwnd` a 1 MSS; (4) en las rondas siguientes la ventana crece a 2 y 4 MSS, alcanza el umbral y pasa a congestion avoidance, donde 4 ACKs que suman 1/4 de MSS cada uno la llevan a 5 MSS; (5) un timeout en congestion avoidance fija `ssthresh` $= \lfloor 35/2 \rfloor = 17$ y vuelve a slow start.

5.2. Go-Back-N sin pérdidas

Los tests de la sección "Test send y recv" de la actividad anterior, más el Test 4 de `buff_size` no múltiplo de 16 y el cierre de conexión, se adaptaron para usar `mode="go_back_n"` (`test_client_gbn.py` y `test_server_gbn.py`). Con la versión de la parte 2 (ventana fija de 8 segmentos de 16 bytes) se aprobaron todos:

```
$ python3 test_server_gbn.py
Test 1 received: b'Mensje de len=16'
Test 1: Passed
Test 2 received: b'Mensaje de largo 19'
Test 2: Passed
Test 3 received: b'Mensaje de largo 19'
Test 3: Passed
Test 4 recieved:  b'aaaaaaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbb'
Test 4: Passed
```

Los mismos tests se aprobaron también con la versión final, con y sin control de congestión.

5.3. Go-Back-N con pérdidas

Con `netem` activo y el modo debug, los cuatro tests se aprobaron nuevamente. La traza del servidor fue:

```
debug:  conexion establecida
Test 1 received: b'Mensje de len=16'
Test 1: Passed
Test 2 received: b'Mensaje de largo 19'
Test 2: Passed
debug:  dato repetido, reenviando ACK 65
debug:  dato repetido, reenviando ACK 65
debug:  dato repetido, reenviando ACK 67
Test 3 received: b'Mensaje de largo 19'
Test 3: Passed
debug:  dato repetido, reenviando ACK 86
Test 4 recieved:  b'aaaaaaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbb'
Test 4: Passed
debug:  TIMEOUT esperando ultimo ACK, reenviando FIN+ACK
debug:  conexion cerrada
```

y la del cliente:

```
debug:  conexion establecida
debug:  TIMEOUT, reenviando ventana
debug:  TIMEOUT, reenviando ventana
debug:  TIMEOUT, reenviando ventana
debug:  TIMEOUT, reenviando ventana
```

```
debug:  TIMEOUT, reenviando ventana
debug:  TIMEOUT, reenviando ventana
debug:  conexion cerrada
OK
```

La traza muestra el funcionamiento de Go-Back-N. El mensaje del Test 3 comienza en el byte 65 y ocupa tres segmentos: el largo (`b"19"`, seq 65), 16 bytes (seq 67) y 3 bytes (seq 83). Se perdió el primero, y los otros dos llegaron fuera de orden, por lo que el receptor los descartó y respondió dos veces con el ACK acumulativo 65. Al cumplirse el timeout, el emisor reenvió la ventana completa; en ese reenvío llegó el segmento 65, pero se volvió a perder el 67, lo que explica el `ACK 67` repetido. En el Test 4 ocurrió lo mismo con el segmento de seq 86, que es justo donde termina el mensaje anterior ($65 + 2 + 19 = 86$). El emisor registró 6 timeouts, más que las pérdidas visibles en el receptor, porque un timeout también ocurre cuando se pierde un ACK, y en ese caso el receptor no detecta nada. También se recuperó una pérdida durante el cierre. Pese a todo, **todos los mensajes llegaron íntegros**.

5.4. Control de congestión sin pérdidas

Para observar el control de congestión con más eventos se usó un archivo de 1215 bytes (`archivo_grande.txt`), que con MSS = 8 ocupa 153 segmentos (el largo y 152 de datos), enviado con `cliente_gbn.py` y `servidor_gbn.py`. Sin `netem`, el inicio y el final de la traza del cliente fueron:

```
debug:  conexion establecida
debug:  [CC] inicio       estado=slow start          cwnd=  8.00 ssthresh=None window_size=1
debug:  [CC] ack 64       estado=slow start          cwnd= 16.00 ssthresh=None window_size=2
debug:  [CC] ack 72       estado=slow start          cwnd= 24.00 ssthresh=None window_size=3
debug:  [CC] ack 80       estado=slow start          cwnd= 32.00 ssthresh=None window_size=4
...
debug:  [CC] ack 1264     estado=slow start          cwnd=1216.00 ssthresh=None window_size=152
debug:  [CC] ack 1272     estado=slow start          cwnd=1224.00 ssthresh=None window_size=153
debug:  [CC] ack 1279     estado=slow start          cwnd=1232.00 ssthresh=None window_size=154
control de congestion: on
segmentos enviados: 153
debug:  conexion cerrada
```

Se enviaron exactamente 153 segmentos, uno por cada segmento del archivo, y la traza no contiene ningún timeout: **no se observaron timeouts espontáneos**. Como nunca hubo un timeout, `ssthresh` permaneció en `None` y toda la transmisión ocurrió en slow start. La ventana comenzó en 1 y creció en 1 MSS por cada ACK, lo que equivale a duplicarse en cada ronda, hasta alcanzar 154 segmentos, más que el archivo completo. Esto muestra que, sin un umbral, slow start no tiene límite: en una red real la ventana crecería hasta saturar algún enlace y provocar pérdidas, que es precisamente el evento que fija el primer `ssthresh`. En `localhost` no existe ese cuello de botella.

Con los tests de la sección anterior, que envían mensajes cortos, se observa además que cada mensaje parte de nuevo con `window_size=1`, ya que se crea un `CongestionControl` por cada `send`.

5.5. Control de congestión con pérdidas

Con `netem` activo, el archivo de 1215 bytes llegó íntegro, lo que se verificó con `cmp`:

```
$ python3 cliente_gbn.py localhost 8000 on < archivo_grande.txt | tee
```

```
    traza_cc_perdidas.txt
...
control de congestion: on
segmentos enviados: 240
$ cmp archivo_grande.txt recibido.txt && echo IDENTICO
IDENTICO
```

Se enviaron 240 segmentos para 153 segmentos de datos, es decir, 87 retransmisiones, y ocurrieron 31 timeouts durante el envío de datos. La figura 2 muestra la evolución de `cwnd` y `ssthresh` (en MSS) a lo largo de los 172 eventos registrados por el modo debug.

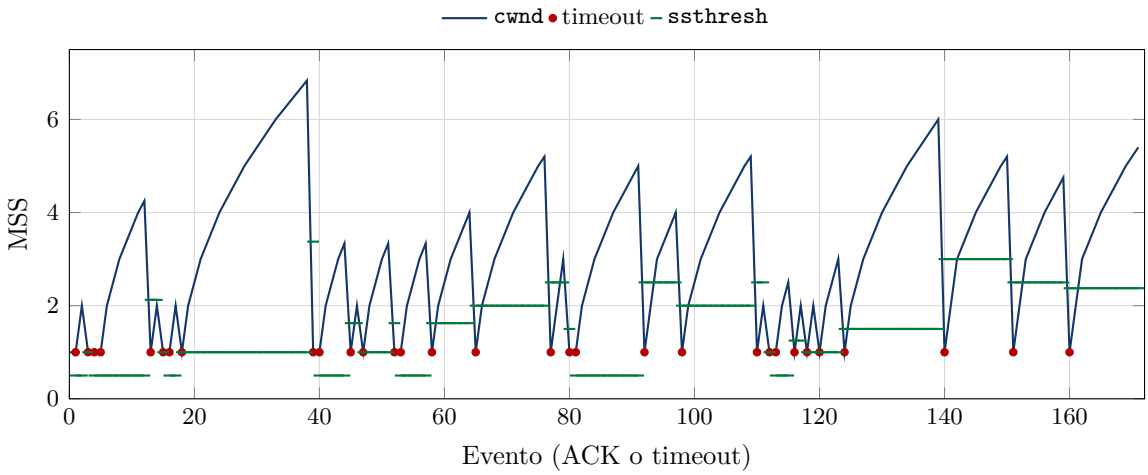


Figura 2: Evolución de `cwnd` con 20 % de pérdida (traza real del envío de 1215 bytes)

El gráfico tiene la forma de "diente de sierra" característica de TCP: la ventana crece y, ante cada timeout, cae a 1 MSS. Con 20 % de pérdida en cada dirección, los timeouts son tan frecuentes que `cwnd` nunca supera los 7 MSS. El siguiente tramo de la traza muestra un ciclo completo:

```
debug:  [CC] ack 1033     estado=congestion avoidance cwnd= 48.00 ssthresh=12.0 window_size=6
debug:  TIMEOUT, reenviando ventana
debug:  [CC] timeout      estado=slow start           cwnd=  8.00 ssthresh=24.0 window_size=1
debug:  [CC] ack 1041     estado=slow start           cwnd= 16.00 ssthresh=24.0 window_size=2
debug:  [CC] ack 1049     estado=congestion avoidance cwnd= 24.00 ssthresh=24.0 window_size=3
debug:  [CC] ack 1057     estado=congestion avoidance cwnd= 26.67 ssthresh=24.0 window_size=3
debug:  [CC] ack 1065     estado=congestion avoidance cwnd= 29.33 ssthresh=24.0 window_size=3
debug:  [CC] ack 1089     estado=congestion avoidance cwnd= 32.00 ssthresh=24.0 window_size=4
```

1. Un timeout ocurre con `cwnd` $= 48$: se fija `ssthresh` $= 48/2 = 24$, `cwnd` vuelve a 1 MSS (8 bytes) y el estado vuelve a slow start. La ventana cae de 6 a 1 segmento.
2. En slow start, cada ACK suma 1 MSS completo: $8 \to 16$.
3. Al llegar a `cwnd` $= 24 \geq$ `ssthresh`, el estado cambia a congestion avoidance.
4. En congestion avoidance cada ACK suma $8 \cdot \frac{1}{3} \approx 2{,}67$, ya que caben 3 MSS en la ventana: $24 \to 26{,}67 \to 29{,}33 \to 32$. Se necesitan 3 ACKs para ganar 1 MSS, y la ventana crece a 4 cuando `cwnd` llega a $32 = 4 \times 8$.

Entre `ack 1065` y `ack 1089` el ACK avanzó 24 bytes de una vez, es decir, tres segmentos: se perdieron ACKs intermedios y, como el ACK es acumulativo, el siguiente confirmó todo

lo anterior. En ese momento la ventana se deslizó varias posiciones con un solo ACK. La traza también registró el caso borde del handshake (`SYN+ACK repetido, reenviando ACK del handshake`), resuelto dentro de `send_using_go_back_n`.

5.6. Comparación con y sin control de congestión

El enunciado pide comparar el tiempo y la cantidad de segmentos enviados por Go-Back-N con y sin control de congestión, con 20 % de pérdida y un timeout menor a 1 segundo. Para ello se escribió `experimento.py`, que en cada corrida levanta un servidor y un cliente en procesos separados, fija `TIMEOUT = 0.5`, mide el tiempo que demora `send` (sin incluir el handshake ni el cierre), registra `number_of_sent_segments` y verifica que el archivo llegue íntegro. Las corridas se alternan entre ambos modos para que los dos se vean afectados por igual si cambian las condiciones del computador.

El enunciado es ambiguo respecto del tamaño del archivo: la pregunta se refiere a un archivo de más de 1000 B, pero las instrucciones indican transmitir uno de 100 B. Por ello el experimento se realizó con ambos tamaños, con cinco corridas por modo en cada caso y la misma configuración de `netem`.

Archivo de 100 B (14 segmentos con MSS = 8):

Cuadro 1: Envío de un archivo de 100 B con 20 % de pérdida, 100 ms de retardo y `TIMEOUT` = 0,5 s

	Con control de congestión		Sin control de congestión	
Corrida	**Tiempo (s)**	**Segmentos**	**Tiempo (s)**	**Segmentos**
1	4,21	28	4,81	46
2	3,11	19	2,61	39
3	7,41	31	2,61	25
4	3,71	20	2,11	32
5	2,81	23	2,11	31
Promedio	**4,25**	**24,2**	**2,85**	**34,6**

En todas las corridas el archivo llegó íntegro. Con control de congestión se enviaron en promedio 24,2 segmentos frente a 34,6 sin él: unas 10 retransmisiones frente a 21, es decir, la mitad. Sin embargo, **el envío sin control de congestión fue más rápido** (2,85 s frente a 4,25 s).

Los tiempos medidos aumentan en escalones de aproximadamente 0,5 s (2,11; 2,61; 3,11; 3,71; 4,21; 4,81), el valor del timeout. Esto indica que **el tiempo total está dominado por la cantidad de timeouts y no por la cantidad de segmentos enviados**: enviar un segmento toma microsegundos, mientras que cada timeout obliga a esperar medio segundo sin enviar nada. Por eso, enviar menos segmentos no implica tardar menos.

La diferencia de tiempos se explica por el tamaño de la ventana. Sin control de congestión, la ventana de 8 segmentos permite enviar los 14 segmentos en dos tandas. Con control de congestión, la ventana parte en 1 y crece de a un segmento por ACK; con un archivo tan pequeño, el envío termina antes de que la ventana alcance a crecer, y cada timeout la devuelve a 1. En cambio, ante un timeout, sin control de congestión se reenvían hasta 8 segmentos, muchos de los cuales el receptor ya había recibido y vuelve a descartar, mientras que con control de congestión se reenvía uno solo. Eso explica la menor cantidad de segmentos.

La corrida 3 con control de congestión (7,41 s) muestra además la alta variabilidad del experi-

mento: una racha de timeouts consecutivos con la ventana en 1 puede duplicar el tiempo. Con solo cinco corridas por modo, los promedios deben interpretarse con cautela.

Archivo de 1215 B (153 segmentos con MSS = 8), que responde directamente a la pregunta del enunciado:

Cuadro 2: Envío de un archivo de 1215 B con 20 % de pérdida, 100 ms de retardo y `TIMEOUT` = 0,5 s

	Con control de congestión		**Sin control de congestión**	
Corrida	**Tiempo (s)**	**Segmentos**	**Tiempo (s)**	**Segmentos**
1	44,29	269	36,08	566
2	36,67	246	29,16	465
3	42,08	265	23,05	409
4	36,18	250	29,16	480
5	47,59	273	25,56	426
Promedio	**41,36**	**260,6**	**28,60**	**469,2**

En todas las corridas el archivo llegó íntegro. Con control de congestión se enviaron en promedio 260,6 segmentos frente a 469,2 sin él, un 44 % menos. Descontando los 153 segmentos del archivo, esto corresponde a unas 108 retransmisiones frente a 316: con control de congestión se retransmitió un tercio de lo que se retransmitió sin él, mientras que sin control de congestión se envió el equivalente a tres veces el archivo completo.

Respondiendo la pregunta del enunciado, **el envío de un archivo de más de 1000 B tomó menos tiempo sin control de congestión**: 28,60 s en promedio frente a 41,36 s, es decir, un 45 % más con control de congestión. La diferencia es consistente: las cinco corridas con control de congestión fueron más lentas que las cinco sin él, ya que la más rápida con control de congestión (36,18 s) superó a la más lenta sin él (36,08 s).

Comparación entre ambos tamaños.

Cuadro 3: Diferencias promedio entre ambos modos según el tamaño del archivo

Archivo	**Segmentos enviados con CC**	**Tiempo con CC**
100 B (14 segmentos)	30 % menos	49 % más
1215 B (153 segmentos)	44 % menos	45 % más

Antes de realizar el experimento con el archivo mayor se esperaba que la desventaja en tiempo del control de congestión disminuyera, ya que la ventana tendría más tiempo para crecer. Los resultados no confirmaron esa expectativa: la penalización en tiempo se mantuvo prácticamente igual. La explicación está en la figura 2: con 20 % de pérdida en cada dirección, los timeouts son tan frecuentes que la ventana con control de congestión nunca supera los 7 segmentos y vuelve a 1 cada pocos ACKs. Un archivo más largo no le da a la ventana la oportunidad de crecer, sino más oportunidades de sufrir pérdidas y volver a empezar. Sin control de congestión, en cambio, siempre hay 8 segmentos en tránsito. La ventaja en cantidad de segmentos, por su parte, sí aumentó con el tamaño del archivo, porque en una transferencia larga se acumulan más timeouts, y en cada uno la diferencia entre reenviar 8 segmentos y reenviar 1 se suma.

Conclusión del experimento. El control de congestión no está diseñado para que una transferencia individual termine antes, sino para **reducir la carga sobre la red** cuando esta

muestra señales de congestión: ante pérdidas, el emisor se frena y envía menos. Si todos los emisores reenviaran ventanas completas ante cada pérdida, una red congestionada recibiría aún más tráfico y se congestionaría más. Al reducir su tasa, cada flujo deja capacidad disponible para los demás, lo que hace que la red sea compartida de forma justa.

6. Situaciones de borde y limitaciones

`ssthresh` menor a un MSS. Cuando ocurren dos timeouts seguidos, el segundo ocurre con `cwnd` $= 1$ MSS, por lo que `ssthresh` $= 8/2 = 4$ bytes, menos que un segmento. Con el primer ACK, `cwnd` $= 16 \geq 4$ y se pasa directamente a congestion avoidance: slow start dura un solo ACK. Esto aparece varias veces en la traza con pérdidas:

```
debug:  [CC] ack 73       estado=congestion avoidance cwnd= 16.00 ssthresh=4 window_size=2
debug:  TIMEOUT, reenviando ventana
debug:  [CC] timeout      estado=slow start           cwnd=  8.00 ssthresh=8 window_size=1
debug:  TIMEOUT, reenviando ventana
debug:  [CC] timeout      estado=slow start           cwnd=  8.00 ssthresh=4 window_size=1
debug:  TIMEOUT, reenviando ventana
debug:  [CC] timeout      estado=slow start           cwnd=  8.00 ssthresh=4 window_size=1
debug:  [CC] ack 81       estado=congestion avoidance cwnd= 16.00 ssthresh=4 window_size=2
```

El comportamiento sigue la regla `ssthresh = cwnd/2` del enunciado y es el que verifica `CongestionControl_test.py`, por lo que se mantuvo. En el TCP real, `ssthresh` tiene un mínimo de 2 MSS precisamente para evitar este caso (Allman, Paxson y Blanton, 2009). Con pérdidas frecuentes, este efecto contribuye a que la ventana permanezca pequeña. Se observa también que `ssthresh` aparece a veces como entero (`4`) y a veces como decimal (`24.0`): en congestion avoidance `cwnd` es un `float` y la división entera de un `float` retorna un `float`, lo que no afecta el funcionamiento.

Reducción de la ventana con segmentos en tránsito. Es el caso borde que describe el enunciado (sección 4.7). Se resolvió deslizando la ventana de a una posición mientras su primer segmento esté confirmado, lo que cubre cualquier cantidad de segmentos confirmados fuera de la ventana. Durante el desarrollo, una versión intermedia que contaba los segmentos confirmados sin deslizar la ventana producía un ciclo infinito: el mismo ACK "confirmaba" siempre el mismo segmento y hacía crecer la ventana indefinidamente.

El temporizador cuando no hay segmentos nuevos. Si un ACK confirma parte de la ventana pero no entra ningún segmento nuevo (por ejemplo, al final del mensaje), detener el temporizador dejaría al emisor esperando un ACK sin ningún temporizador activo; si ese ACK se perdiera, `recvfrom` esperaría indefinidamente. Por eso, en ese caso el temporizador sigue corriendo desde el último envío.

Las pérdidas a mano no afectan a Go-Back-N. Como los datos se envían con `SocketUDP.sendto` y no con `send_segment`, `LOSS_RATE` solo afecta a los ACK y a los mensajes del handshake y del cierre. Por ello las pruebas con pérdidas se hicieron con `netem`. Para simular pérdidas a mano en los datos habría que descartar el envío sin dejar de iniciar el temporizador, ya que de lo contrario un segmento "perdido" nunca provocaría un timeout.

Espera activa de `SocketUDP`. `SocketUDP.recvfrom` usa un socket no bloqueante y revisa continuamente si llegó algo, por lo que el emisor ocupa un núcleo del procesador mientras espera. No afecta la corrección, pero sí el consumo de CPU.

Un control de congestión por mensaje. Al crear un `CongestionControl` por cada `send`, la información aprendida sobre la red se pierde entre mensajes: cada mensaje vuelve a empezar

con una ventana de 1. En los tests con mensajes cortos, la ventana nunca supera unos pocos segmentos.

Slow start sin límite. Sin pérdidas, `ssthresh` nunca se define y la ventana crece hasta superar el tamaño del archivo (154 segmentos en la prueba sin pérdidas).

7. Conclusiones

Se implementó Go-Back-N sobre la versión simplificada de TCP de la actividad anterior, y luego un control de congestión basado en slow start y congestion avoidance de TCP Tahoe. En todas las pruebas, con y sin pérdidas, los mensajes y archivos llegaron íntegros, incluido un archivo de 1215 bytes con 20 % de pérdida en ambas direcciones.

La implementación mostró que Go-Back-N es una generalización directa de Stop & Wait. El receptor de la actividad anterior no requirió ningún cambio, porque ya confirmaba con un ACK acumulativo y descartaba los segmentos fuera de orden; todo el trabajo estuvo en el emisor, que debe manejar la ventana, un único temporizador y la retransmisión de la ventana completa ante un timeout.

La clase `CongestionControl`, al ser una máquina de estados independiente de los sockets, pudo probarse por separado y luego integrarse en el emisor simplemente informándole los eventos de ACK y timeout. Las trazas del modo debug permitieron observar el comportamiento esperado de TCP Tahoe: crecimiento exponencial en slow start, crecimiento lineal en congestion avoidance y la caída a 1 MSS con cada timeout, con la forma de diente de sierra característica.

El experimento de comparación mostró que el control de congestión redujo las retransmisiones a entre la mitad y un tercio, según el tamaño del archivo, pero aumentó el tiempo de envío en cerca de un 45 % en ambos tamaños, ya que ese tiempo está dominado por la cantidad de timeouts y, con pérdidas frecuentes, la ventana con control de congestión permanece pequeña. Esto ilustra que el objetivo del control de congestión no es acelerar una conexión individual, sino proteger a la red y repartir su capacidad de forma justa entre los flujos que la comparten.

Las extensiones naturales de este trabajo serían fast retransmit, que detecta pérdidas con tres ACK duplicados sin esperar el timeout, mantener la ventana de congestión durante toda la conexión, y Selective Repeat, que evitaría retransmitir segmentos que el receptor ya recibió.

Referencias

Allman, M., Paxson, V. y Blanton, E. (2009). *TCP congestion control (rfc 5681)*. Internet Engineering Task Force. Descargado de https://www.rfc-editor.org/rfc/rfc5681 (Recuperado el 18 de octubre de 2026)

Equipo docente CC4303. (2026). *Actividad: Go-Back-N y control de congestión*. Departamento de Ciencias de la Computación, Universidad de Chile. Plataforma EOL. (Recuperado el 18 de octubre de 2026)

Jacobson, V. (1988). Congestion avoidance and control. *ACM SIGCOMM Computer Communication Review*, 18(4), 314–329.

Kurose, J. F. y Ross, K. W. (2021). *Computer networking: A top-down approach* (8.ª ed.). Boston: Pearson.

Linux man-pages. (2026). *tc-netem(8) — Network Emulator*. Descargado de https://man7.org/linux/man-pages/man8/tc-netem.8.html (Recuperado el 18 de octubre de 2026)